

Session

648bd223

Agent pipeline

1. **Planner** — refines your question
2. **Document Reader** — searches local guide
3. **Web Enricher** — Google Serper live search
4. **Synthesizer** — combines sources (Groq)
5. **Report Writer** — formats report (Gemini)

Try these

What are the biggest LLM security risks?
How does RAG reduce hallucinations?
What is LLM-as-Judge evaluation?
Best practices for deploying LLM agents?

Document Intelligence Agent

LangGraph · LangSmith · Groq · Gemini · Google Serper



what is THE LLM PRODUCTION STACK



Overview of a Typical Large Language Model (LLM) Production Stack

A typical LLM production stack involves several components and technologies, including data preprocessing, model training, deployment, and maintenance. The following sections provide a detailed analysis of each stage.

Data Preprocessing

The data preprocessing stage is critical in preparing the data for model training. This stage involves:

- **Ingestion Pipeline:** Converts raw documents (PDFs, web pages, databases) into vector embeddings stored in a vector database [1](#).
- **Chunk Size:** The chunk size is critical, with typical sizes ranging from 256 to 1024 tokens and a 10-20% overlap to avoid cutting sentences at boundaries [1](#).
- **Data Pipelines:** Tools like Apache Airflow and Prefect manage ingestion, preprocessing, and transformation of data from raw sources [2](#).

Model Training

The model training stage involves:

- **Neural Network Layers:** Embedding, attention, and feedforward layers work together to process the input text [2](#).
- **Machine Learning Pipeline:** This pipeline is responsible for training the LLM using the preprocessed data.

Deployment

The deployment stage involves:

- **Gateway Layer:** Routes and provides resilience for the LLM application [1](#).
- **Guardrails Layer:** Enforces safety and compliance policies on every request and response, using tools like NVIDIA NeMo Guardrails and Colang [1](#).
- **Evaluation Layer:** Measures the quality of the LLM application [1](#).
- **Observability Layer:** Monitors and debugs the LLM application [1](#).
- **Helm Chart:** Installs the standard vLLM production stack [2](#).
- **MLflow:** Provides tools and APIs for managing the model lifecycle, making it easy to deploy and administer LLMs in a production setting [2](#).

Maintenance

The maintenance stage involves:

- **Monitoring Frameworks:** Monitors the LLM application for performance and issues [2](#).
- **Governance Controls:** Ensures compliance with safety and compliance policies [2](#).
- **Optimization Mechanisms:** Optimizes the LLM application for better performance [2](#).

In conclusion, a typical LLM production stack involves a range of components and technologies, from data preprocessing to deployment and maintenance. By understanding these components and technologies, developers can build and deploy effective LLM applications.

References:

Footnotes

1. DOCUMENT KNOWLEDGE [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#)
2. WEB SEARCH RESULTS [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#) [↩⁷](#)

Pipeline details



what are LLM SECURITY THREATS



Security Threats and Vulnerabilities in Large Language Models (LLMs)

Large Language Models (LLMs) are powerful tools with various applications, but they also introduce specific security threats and vulnerabilities. According to the OWASP Top 10 for LLM Applications, deploying language models poses critical security risks.

Types of Security Threats

The following are some of the specific security threats and vulnerabilities associated with LLMs:

- **Data Poisoning:** This occurs when an adversary injects malicious data into the training phase of an LLM, compromising its performance and reliability [1].
- **Model Inversion Attacks:** These attacks involve reverse-engineering an LLM to extract sensitive information, such as intellectual property or personal data [2].

- **Adversarial Attacks:** These attacks exploit weaknesses in an LLM to manipulate its output, potentially leading to fraudulent or malicious activities [3].
- **Prompt Injection:** This involves crafting malicious prompts to manipulate an LLM's output, potentially leading to security breaches or unauthorized access to data [4].
- **Backdoor Attacks:** These attacks involve inserting malicious code or data into an LLM, allowing an adversary to access or manipulate the model's output [5].
- **Jailbreaking:** This involves exploiting vulnerabilities in an LLM to bypass security controls or access restricted functionality [6].

Defense Techniques

To mitigate these security threats, it is essential to implement defense techniques, such as:

- **Validating LLM Output:** Treating LLM output as untrusted user input and validating it before passing it to downstream systems [7].
- **Sanitizing Input:** Sanitizing user input to prevent malicious data from being injected into an LLM [8].
- **Implementing Access Controls:** Implementing access controls to restrict access to sensitive data and functionality [9].
- **Monitoring and Auditing:** Monitoring and auditing LLM activity to detect and respond to potential security threats [10].

Conclusion

LLMs introduce specific security threats and vulnerabilities, including data poisoning, model inversion attacks, and adversarial attacks. To mitigate these risks, it is essential to implement defense techniques, such as validating LLM output, sanitizing input, and implementing access controls. By understanding these security threats and vulnerabilities, organizations can take proactive steps to protect their LLMs and prevent potential security breaches.

References:

- [1] Jaffal, N. (2025). The subcategories include backdoor attacks, data poisoning, prompt injection, and jailbreaking, each paired with the corresponding defense techniques.
- [2] Models can be reverse-engineered or stolen, leading to intellectual property and privacy breaches.
- [3] Attackers can use LLMs to analyze known vulnerabilities and craft malicious prompts.
- [4] This study comprehensively examines two critical security vulnerabilities in LLMs: prompt injection and data poisoning attacks.
- [5] The subcategories include backdoor attacks, data poisoning, prompt injection, and jailbreaking, each paired with the corresponding defense techniques.
- [6] Jailbreaking: This involves exploiting vulnerabilities in an LLM to bypass security controls or access restricted functionality.
- [7] LLM02 — Insecure Output Handling occurs when LLM responses are passed to downstream systems without validation.
- [8] Sanitizing user input to prevent malicious data from being injected into an LLM.
- [9] Implementing access controls to restrict access to sensitive data and functionality.
- [10] Monitoring and auditing LLM activity to detect and respond to potential security threats.

Pipeline details



Ask anything about LLM production...

